

Chapter 2C: Uninformed Search (Part II)

AMCS2104 Fundamentals of Artificial Intelligence

Content

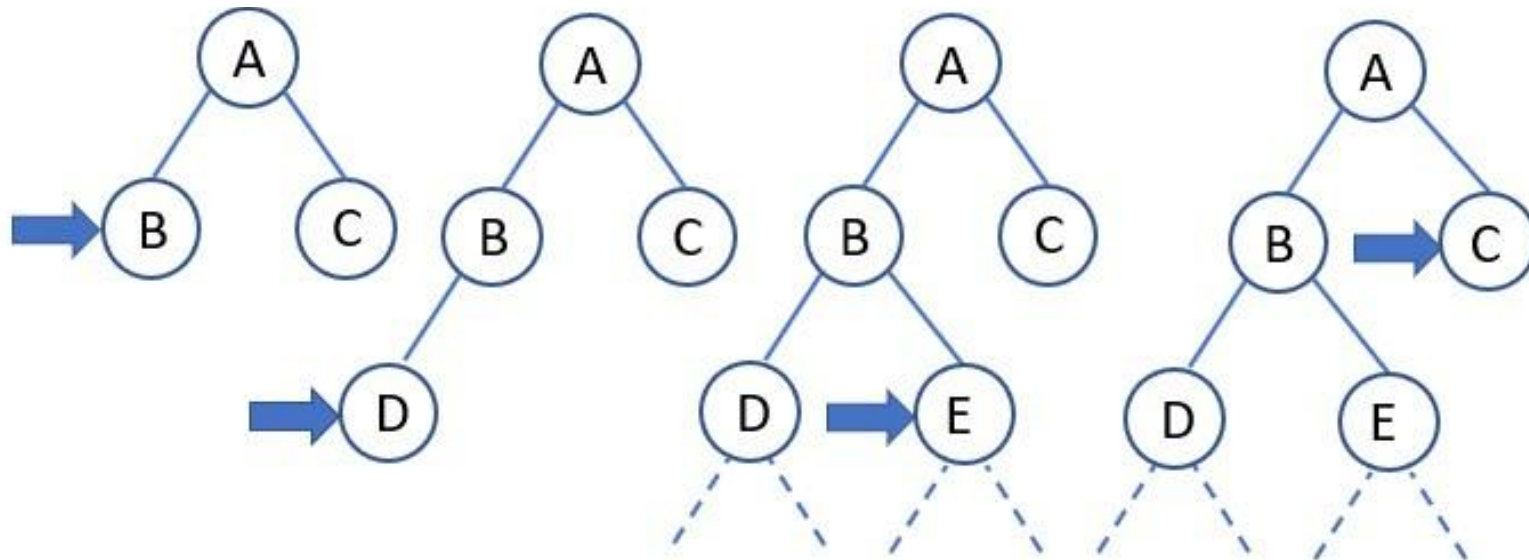
- Backtracking Search
- Depth-Limited Search
- Iterative Deepening Search
- Bidirectional Search

Backtracking Search

- A variant of depth-first search called **backtracking search** (Chapter 6) uses even lesser memory.
- Only one successor is generated at a time rather than all successors.
- Each partially expanded node remembers which successor to generate next.
- Therefore, it reduces the memory requirements from $O(bm)$ to $O(m)$.

Depth-Limited Search

- The unbounded tree problem happens to appear in the depth-first search can be fixed by **imposing a predetermined limit L to the depth of the search domain.**
- This approach is called **depth-limited search**, a version of depth-first search.
- If no solution is found, it returns a **cutoff** instead of a **failure**.



Depth-Limited Search: Performance Evaluation

- **Completeness**

- Incomplete if $L < d$.

- **Optimality**

- Not cost-optimal if $L > d$.

- **Time complexity**

- Exponential complexity: $O(b^L)$.

- **Space complexity**

- Linear complexity: $O(bL)$.

- **Question:** How to identify a **better limit** that can lead to a more efficient search?

Depth-Limited Search

- ❑ Incorrect choice of L can introduce **incompleteness** and **non-cost-optimal** issues.
- ❑ Sometimes a good L can be chosen based on knowledge of the problem.
- ❑ Example
 - ❑ There are 20 cities in Romania (based on the map).
 - ❑ Therefore, $L = 19$ is a valid limit.
 - ❑ If we studied the map carefully, any city can be reached from any other city in at most 9 actions only.
 - ❑ This number, known as the **diameter** of the state-space graph, gives us a better L , which leads to a more efficient depth-limited search.
- ❑ However, for most problems we will not know a good L until they are solved.

Iterative Deepening Search

- **Iterative deepening search** (or **iterative deepening depth-first search**) is a general strategy, used to find the **diameter**.
- It increases the L gradually until it reaches d , the depth of the shallowest goal node.
- If a solution is not found, it returns a **failure** value rather than the cutoff value.

Iterative Deepening Search: Algorithm

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution node or *failure*
 for *depth* = 0 **to** ∞ **do**
 result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)
 if *result* $\neq$ *cutoff* **then return** *result*

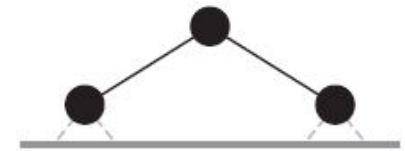
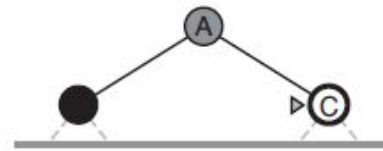
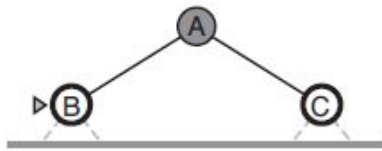
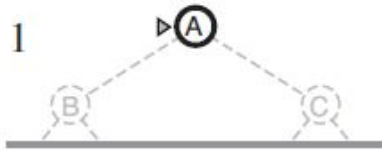
function DEPTH-LIMITED-SEARCH(*problem*, ℓ) **returns** a node or *failure* or *cutoff*
 frontier $\leftarrow$ a LIFO queue (stack) with NODE(*problem*.INITIAL) as an element
 result $\leftarrow$ *failure*
 while not IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node*
 if DEPTH(*node*) > ℓ **then**
 result $\leftarrow$ *cutoff*
 else if not IS-CYCLE(*node*) **do**
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 add *child* to *frontier*
 return *result*

Iterative Deepening Search

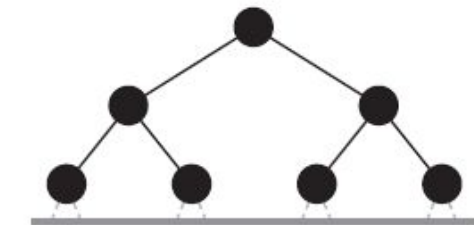
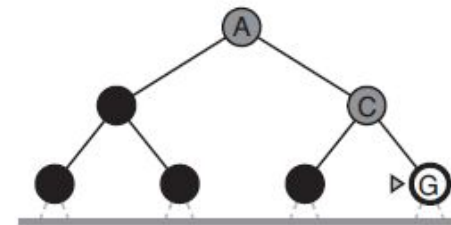
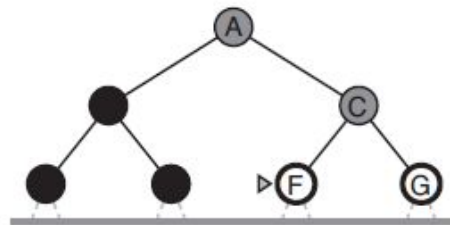
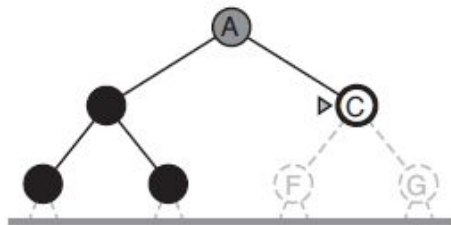
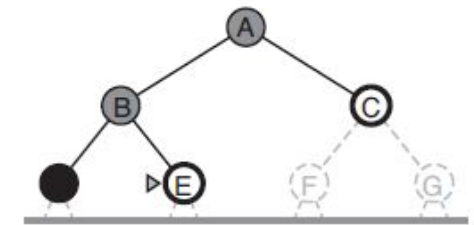
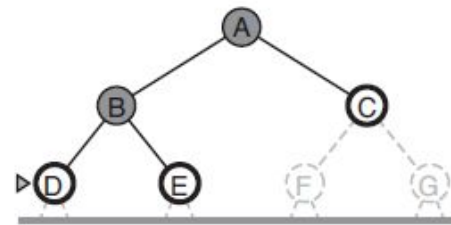
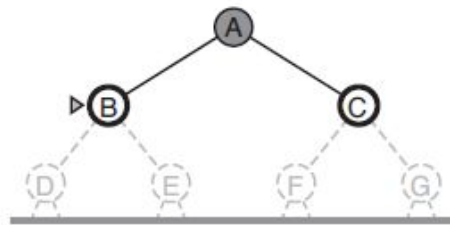
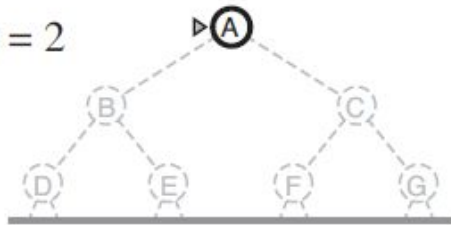
Limit = 0



Limit = 1

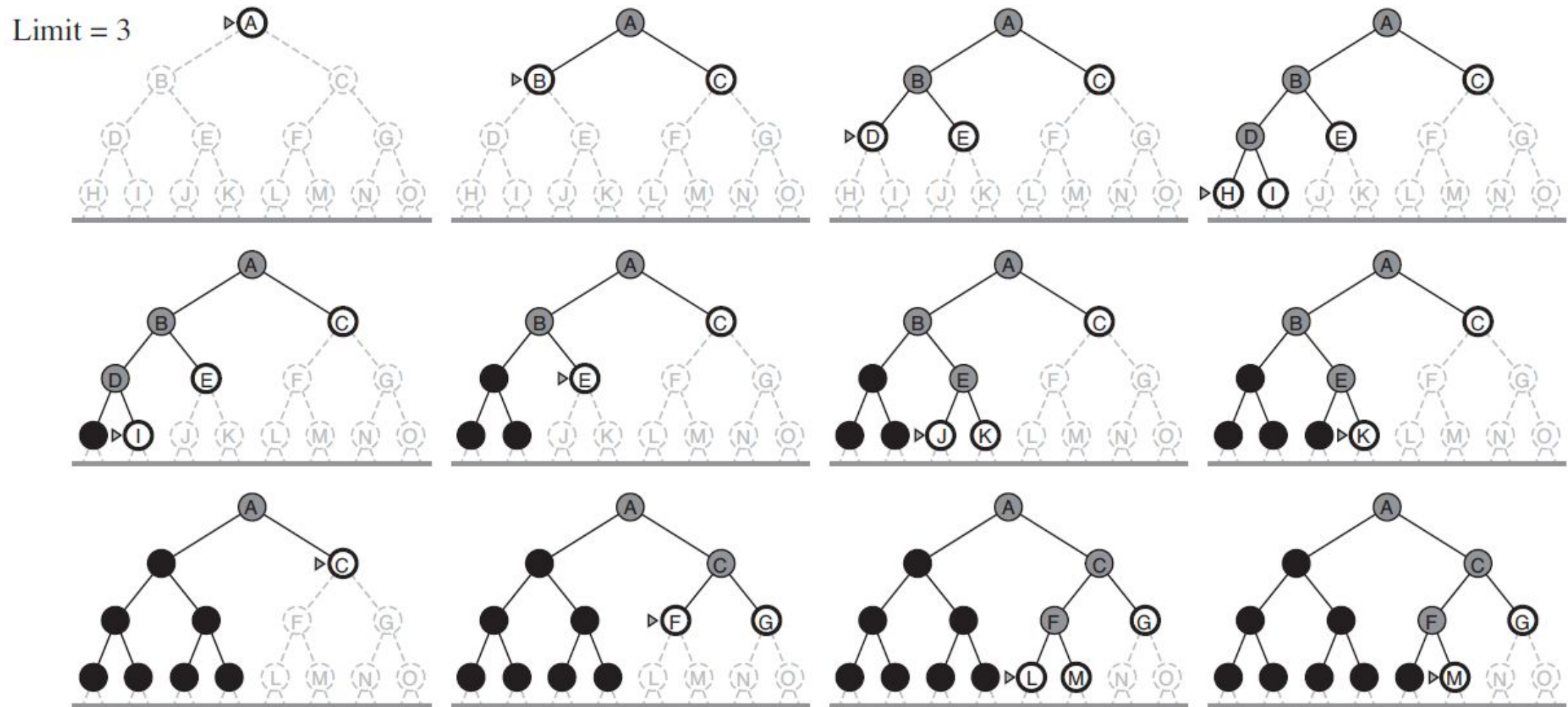


Limit = 2



Four iterations of iterative deepening search on a binary tree.

Iterative Deepening Search



Four iterations of iterative deepening search on a binary tree.

Iterative Deepening Search: Strengths

- **Iterative deepening search** combines the benefits of depth-first search and breadth-first search:
 - Depth-first search
 - Space complexity (if a solution exists on a finite state space): $O(bd)$.
 - Space complexity (if a solution doesn't exist on a finite state space): $O(bm)$.
 - Complete on finite acyclic state spaces.
 - Complete on finite cyclic state spaces if it checks nodes for **cycles** all the way up the path.
 - Breadth-first search
 - Optimal if the all actions have the same cost.

Iterative Deepening Search: Performance Evaluation

□ Completeness

- Complete on finite acyclic state spaces.
- Complete on finite cyclic state spaces if it checks nodes for **cycles** all the way up the path.

□ Optimality

- Cost-optimal if the all actions have the same cost.

□ Time complexity

- Exponential complexity: $O(b^d)$ on finite state spaces with a solution.
- Exponential complexity: $O(b^m)$ on finite state spaces with no solution.

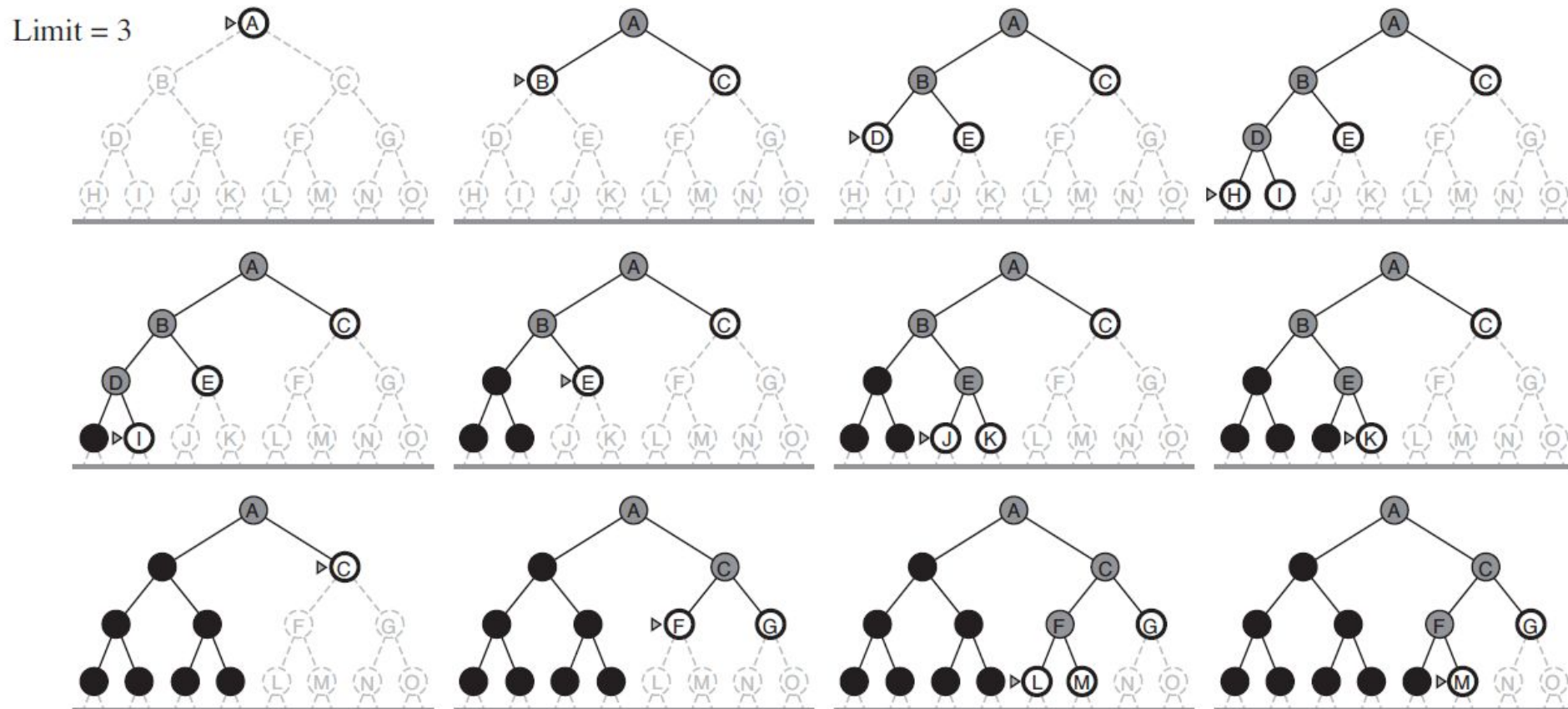
Iterative Deepening Search: Performance Evaluation

□ Space complexity

- Linear complexity: $O(bd)$ on finite state spaces with a solution.
- Linear complexity: $O(bm)$ on finite state spaces with no solution.

Iterative Deepening Search: Efficiency

□ Question: Will iterative deepening search result in a waste of resources?



Iterative Deepening Search: Efficiency

- Assumption:
 - Branching factor is (almost) the same at every level.
- The number of nodes generated:
 - Nodes generated once at the bottom-most level is the highest.
 - Nodes generated twice at the second bottom-most level is the second highest.
 - ...
 - Nodes generated $d - 1$ times at the second highest level is second lowest.
 - Nodes generated d times at the top is the lowest.

Iterative Deepening Search: Efficiency

- The total number of nodes generated by **iterative deepening search** (IDS) is

$$(d)b + (d - 1)b^2 + \dots + (1)b^d$$

- The total number of nodes generated by **breadth-first search** (BFS) is

$$1 + b + b^2 + b^3 + \dots + b^d$$

- If **b = 10** and **d = 5**, then

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$

$$N(\text{BFS}) = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$$

- In general, **iterative deepening search** is the **preferred uninformed search method** when the search space is large and the depth of the solution is not known.

Bidirectional Search

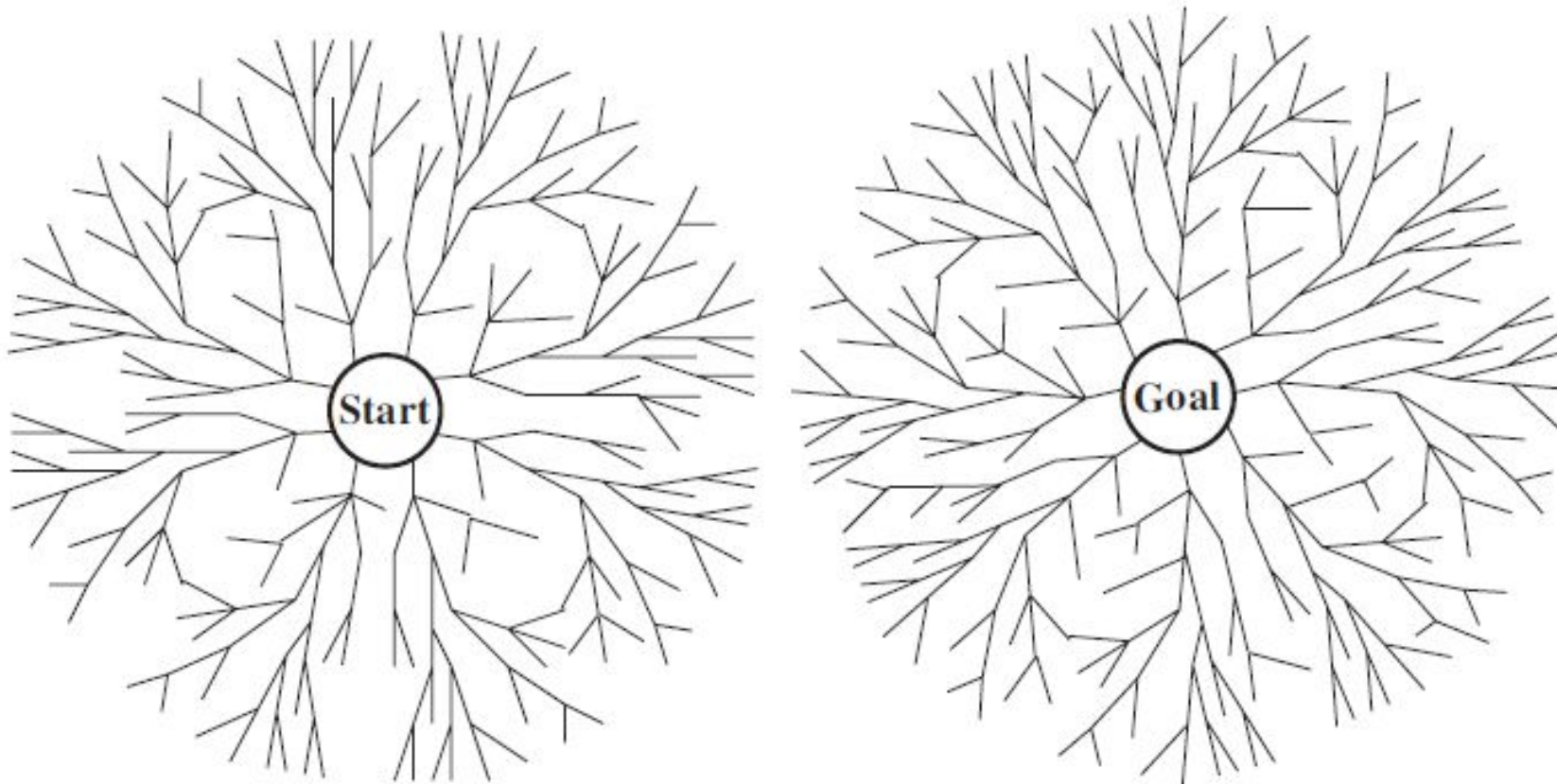
- Given **b = 10** and **d = 6**, the total number of nodes generated by **breadth-first search** is

$$1 + 10 + 100 + 1,000 + 10,000 + 100,000 + 1,000,000 = 1,111,111$$

- The number of nodes generated in the bottom half of the search tree is almost 1000 times greater than the number of nodes generated in the upper half part!
- This can be reduced further roughly by half if one of the two searches is done by iterative deepening.

Bidirectional Search

- A schematic view of a **bidirectional search**.



Bidirectional Search

- Based on the motivation that $b^{d/2} + b^{d/2}$ is very much less than b^d , **bidirectional search** run two simultaneous searches hoping that the two searches meet in the middle:
 - A **forward search** from the initial state.
 - A **backward search** from the goal.
- Then, a way to reason backwards is needed:
 - **Forward search**: state s' is a successor of s in the forward direction
 - **Backward search**: s is a successor of s' .
- A solution will be found when the two frontiers collide or crash into each other.

Bidirectional Search: Performance Evaluation

□ **Completeness**

- **Assumption:** Forward search and backward search are implemented with breadth-first search.
- Complete, provided **b** is finite, where **b** is the branching factor of the tree.

□ **Optimality**

- Cost-optimal, provided **all action costs are the same**

□ **Time complexity**

- Exponential complexity: $O(b^{d/2})$.

□ **Space complexity**

- Linear complexity: $O(b^{d/2})$.

Comparing Uninformed Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

b is the branching factor;

d is the depth of the shallowest solution;

m is the maximum depth of the search tree;

ℓ is the depth limit.

¹complete if b is finite, the state space is finite;

²complete if all action costs $\geq \epsilon > 0$;

³cost-optimal if action costs are identical;

⁴if both directions use breadth-first or uniform-cost search.

Chapter 3E: Informed Search

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Informed Search Strategies
- Best-First Search
- Heuristic Functions
- Greedy Best-First Search
- A* Search
- Admissibility
- Consistency

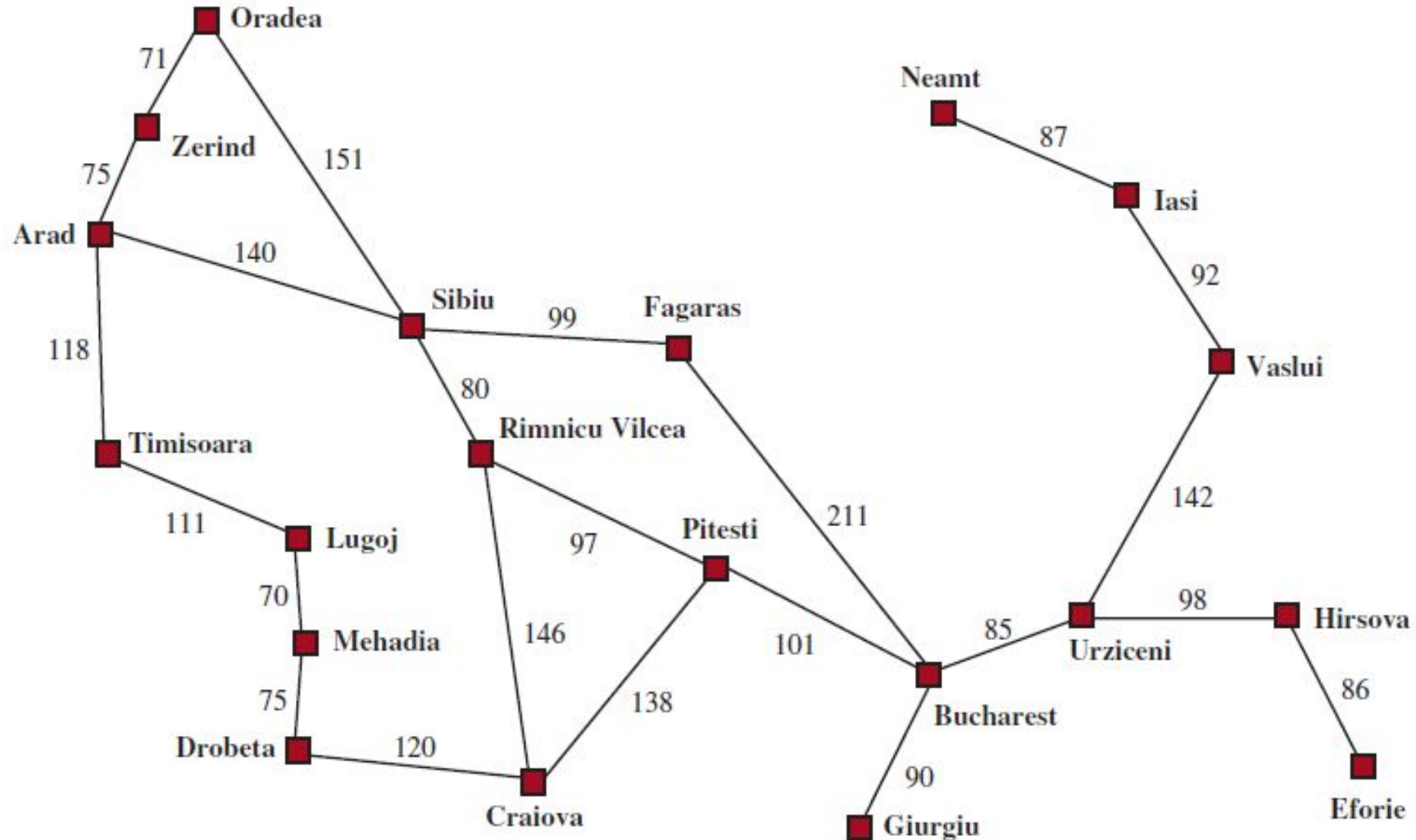
Informed Search Strategies

- Also called **heuristic search**.
- Has additional information about states beyond that provided in the problem definition.
- With this information, it can estimate how far a state is from the goal state.
- Can differentiate between goal state and non-goal state.
- Can identify whether one non-goal state is "more promising" than others.
- In general, informed search strategies can find solutions **more efficiently** than uninformed search strategies.

Best-First Search

- **Best-first search** selects a node for expansion based on an **evaluation function**, $f(n)$.
- All informed search strategies differ in terms of the choice of f -value.
- Node with the lowest f -value (i.e., the best choice) is always expanded first.
- Two best-first search variants that will be covered are:
 - Greedy best-first search
 - A* search

A Simplified Road Map of Part of Romania



Heuristic Functions

- All **best-first search** algorithms include a **heuristic function**.
- A **heuristic** is used to estimate how promising a non-goal state
- A **heuristic** can be represented by a **heuristic function**, denoted as:

$h(n)$ = **estimated cost of the cheapest path** from the state at node n to a goal state

- Example
 - A straight line distance (h_{SLD}) between two points is a heuristic.
- Notice that the values of h_{SLD} cannot be computed from the problem description itself such as the **ACTIONS** and **RESULT** functions.

Heuristic Functions

- Suppose the goal is Bucharest, the straight-line distances to Bucharest is

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

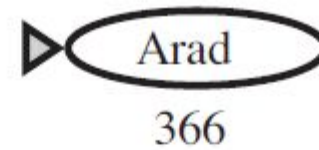
Figure 3.16 Values of h_{SLD} —straight-line distances to Bucharest.

Greedy Best-First Search

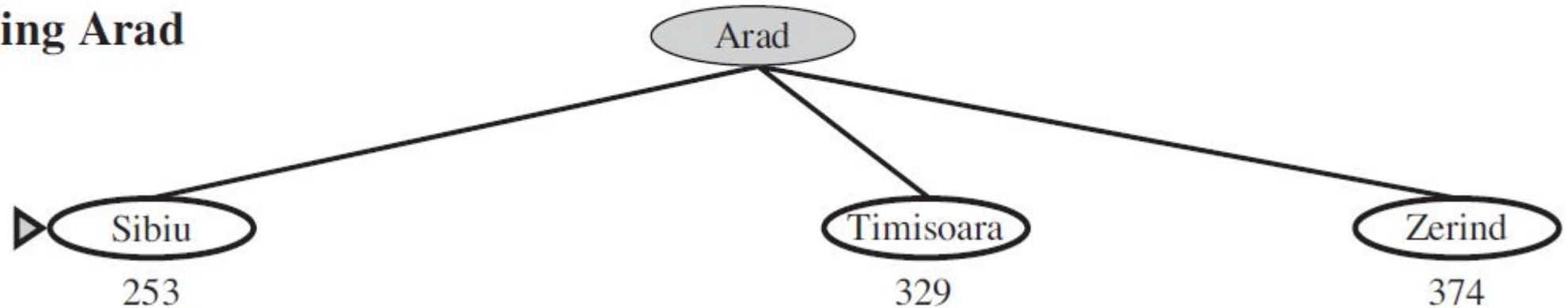
- **Greedy best-first search** tries to expand the node that is closest to the goal which might lead to a solution quickly.
- Hence, it is referred to as a **greedy** strategy.
- It evaluates nodes by using just the heuristic function $f(n) = h(n)$.
- The implementation of greedy best-first search is identical to that for uniform-cost search, except that:
 - **Greedy best-first search**: Order priority queue based on h .
 - **Uniform-cost search**: Order priority queue based on g .

Greedy Best-First Search: Example

(a) The initial state



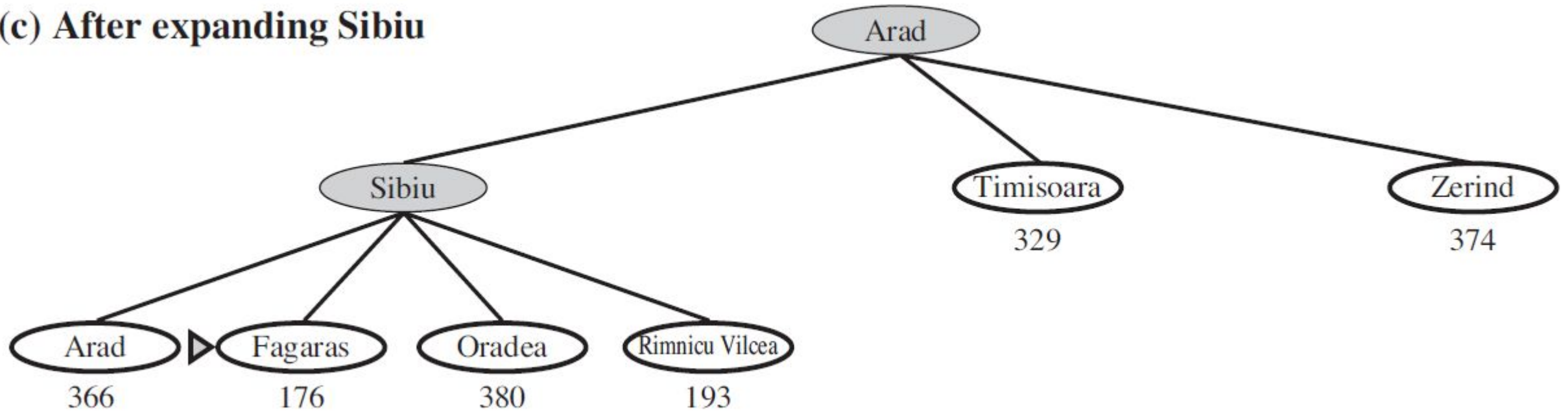
(b) After expanding Arad



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy Best-First Search: Example

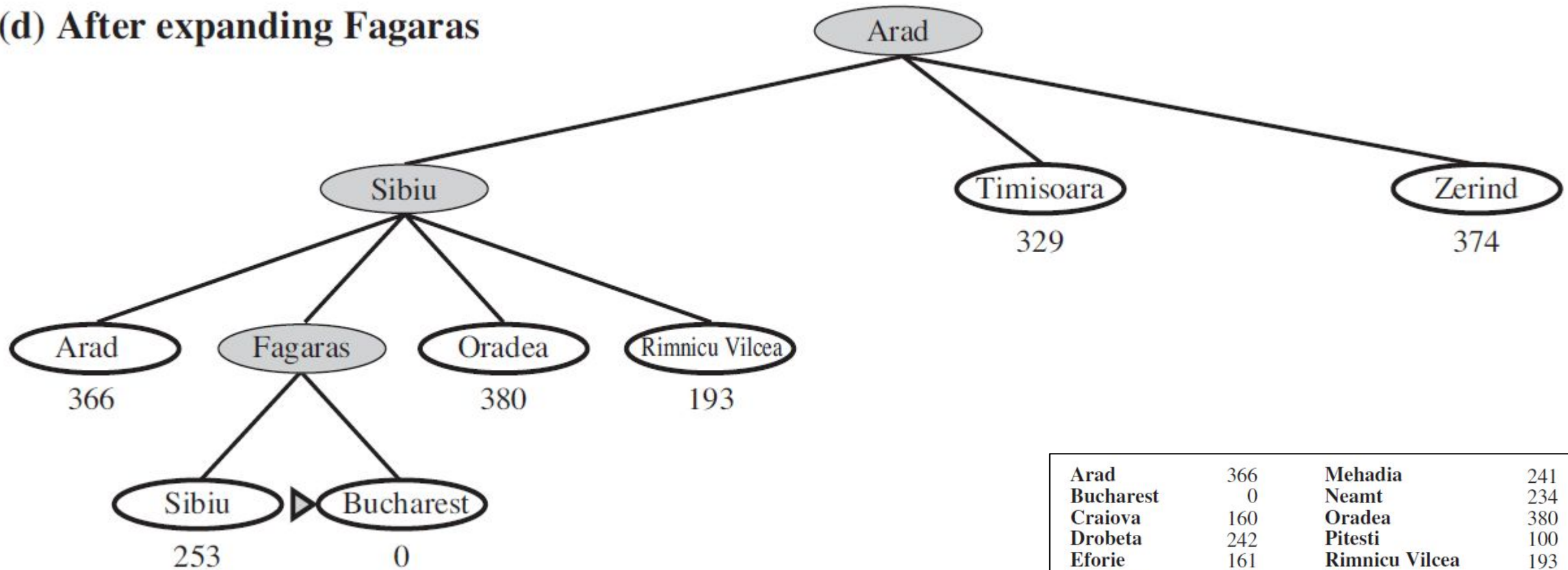
(c) After expanding Sibiu



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy Best-First Search: Example

(d) After expanding Fagaras



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy Best-First Search: Performance Evaluation

□ Completeness

- The tree-like search version is **incomplete** in a finite state space.
- The graph search version is **complete** in a finite state space.

□ Optimality

- Not cost-optimal.

□ Time complexity

- Exponential complexity: $O(b^m)$ for a tree-like search version.
- However, with a good heuristic function, it can be reduced to $O(bm)$.

Greedy Best-First Search: Performance Evaluation

□ Space complexity

- Linear complexity: $O(bm)$ for the tree-like search version.
- Exponential complexity: $O(b^m)$ for the graph search version.

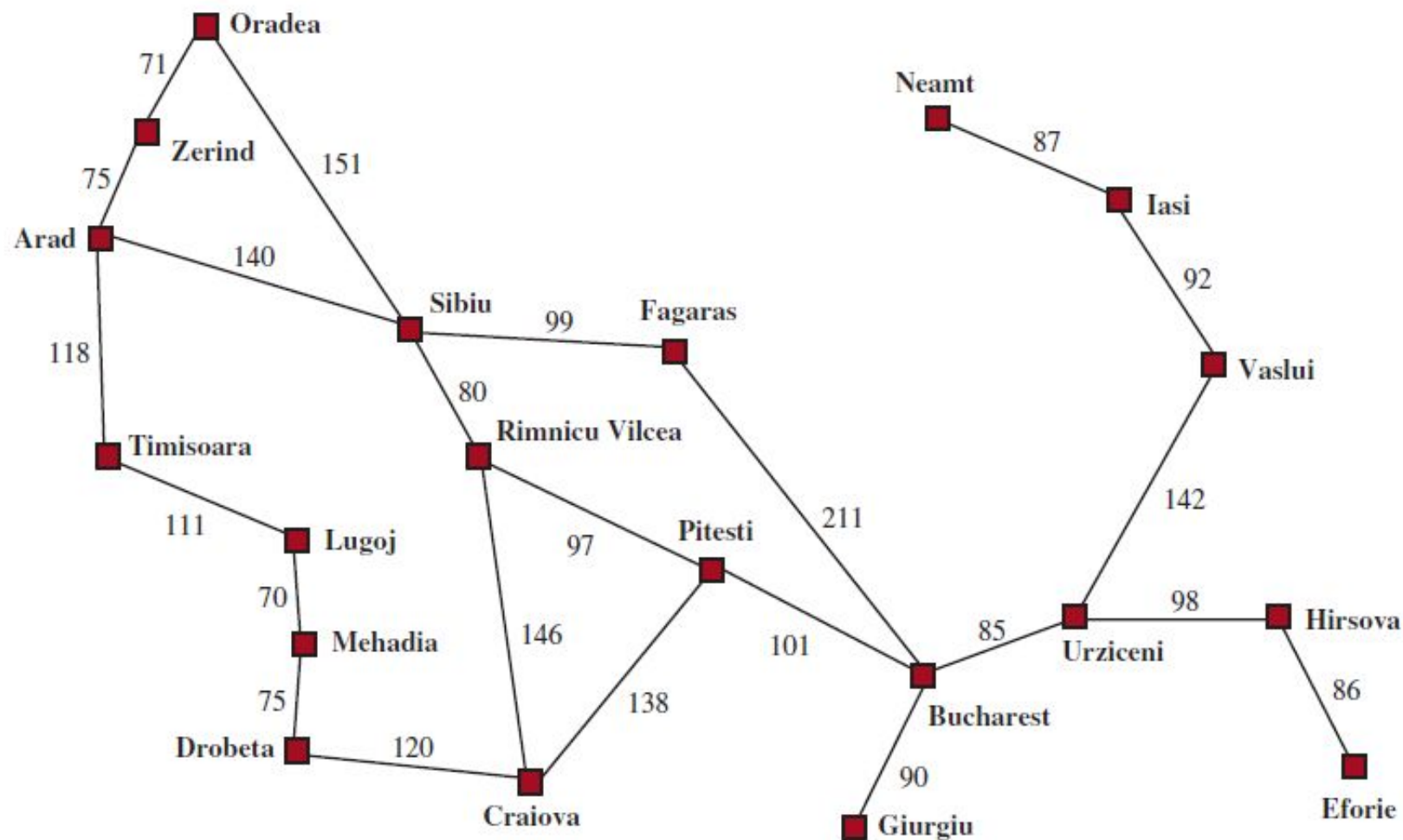
A* Search

- **A* search** (pronounced as A-star search) is the most popular variant of best-first search.
- It evaluates nodes by combining $g(n)$ and $h(n)$ where $g(n)$ is the **path cost** from the root to node n , and $h(n)$ is the **estimated cost** of the **cheapest path** from n to the goal:

$$f(n) = g(n) + h(n)$$

- Thus, its $f(n)$ represents an **estimated cost** of the **cheapest solution** through n .
- The implementation of A* search is identical to that for greedy best-first, except:
 - **A* search**: Order priority queue based on $g + h$.
 - **Greedy best-first search**: Order priority queue based on h .

Information: $g(n)$ and $h(n)$



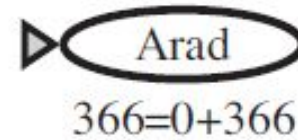
Step cost, $g(n)$

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

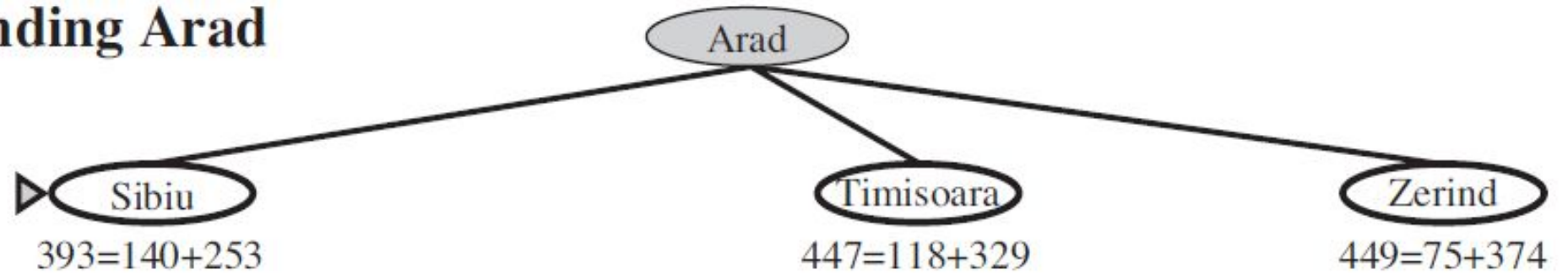
Straight-line distance, $h_{SLD}(n)$

A* Search: Example

(a) The initial state

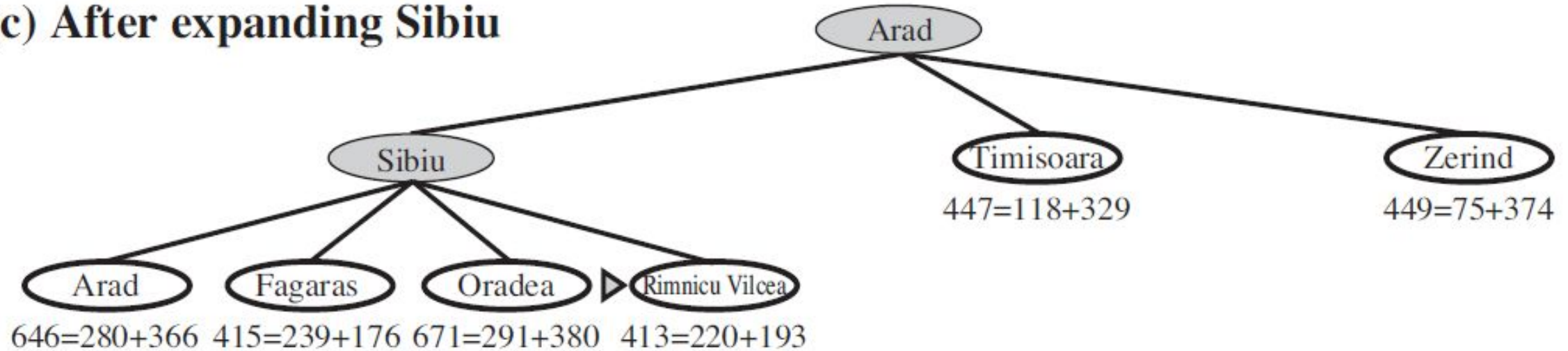


(b) After expanding Arad



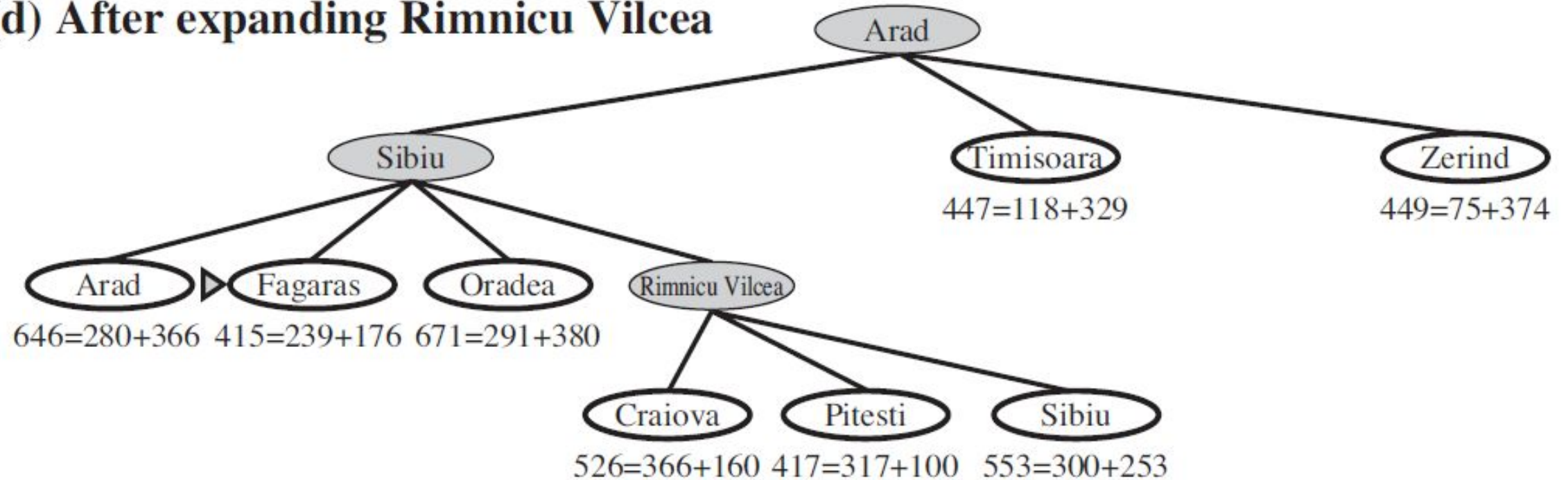
A* Search: Example

(c) After expanding Sibiu



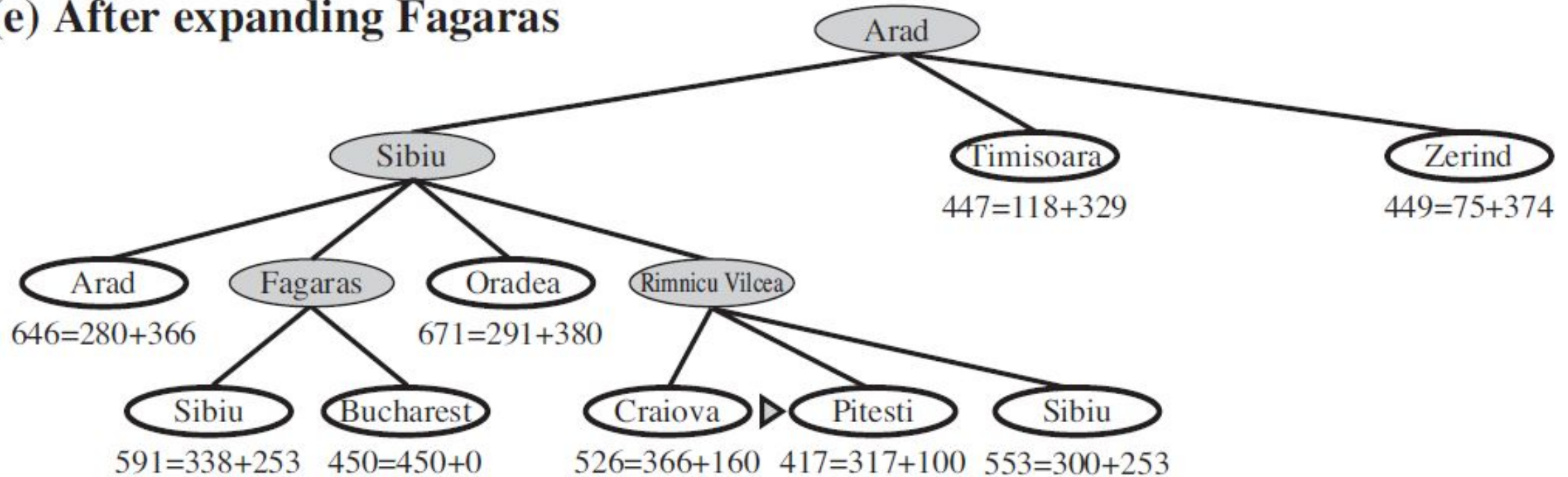
A* Search: Example

(d) After expanding Rimnicu Vilcea



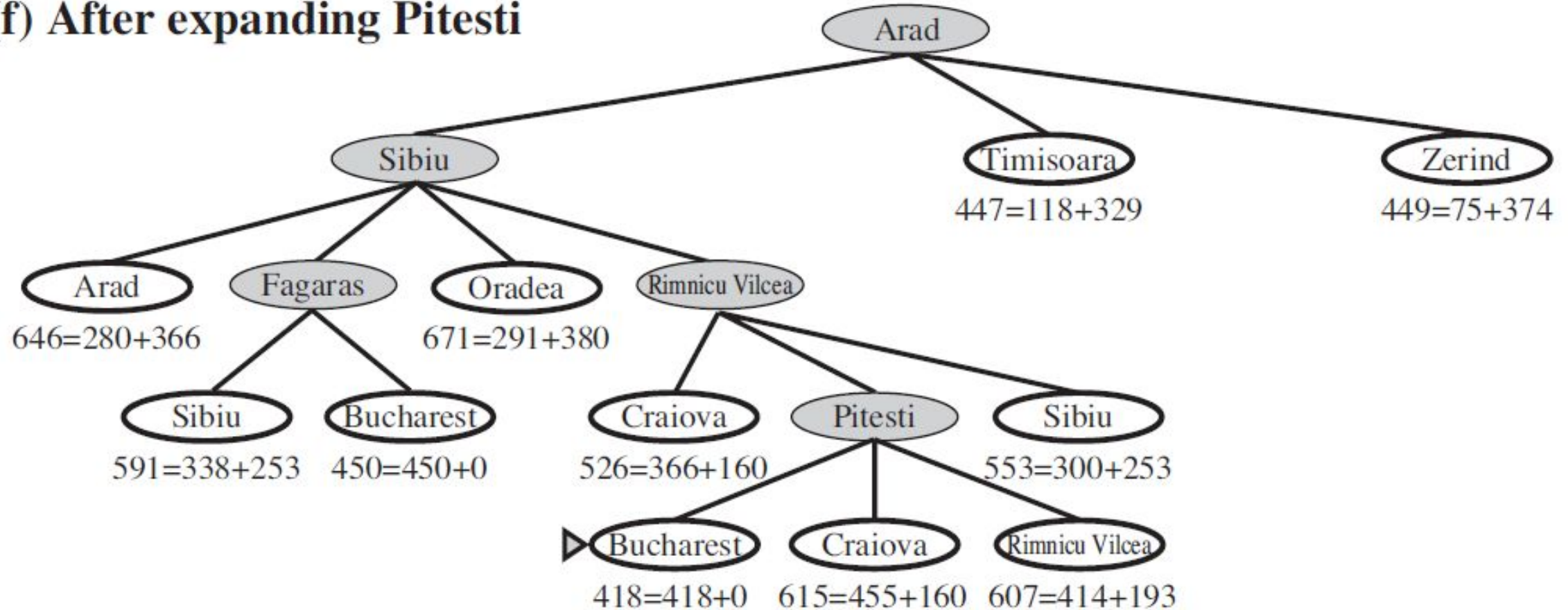
A* Search: Example

(e) After expanding Fagaras



A* Search: Example

(f) After expanding Pitesti



A* Search: Efficiency

- A* is efficient among many other solutions because it **prunes** away search tree nodes that are not necessary for finding an optimal solution.
- Example
 - Timisoara has a f -value of 447 and Zerind has a f -value of 449 (see the figure in the previous slide).
 - Despite being the successors of the root, they are never expanded.
 - The concept of pruning is important for many areas of A.I. as it eliminates possibilities from consideration without having to examine them.
- However, the effectiveness of pruning is dependent on the quality of heuristic used.

A* Search: Performance Evaluation

□ **Completeness**

- Complete if b is finite and the cost of every action exceeds some small positive constant ϵ

□ **Optimality**

- Cost-optimal if the heuristic is admissible.
- Cost-optimal if the heuristic is consistent.

□ **Time complexity**

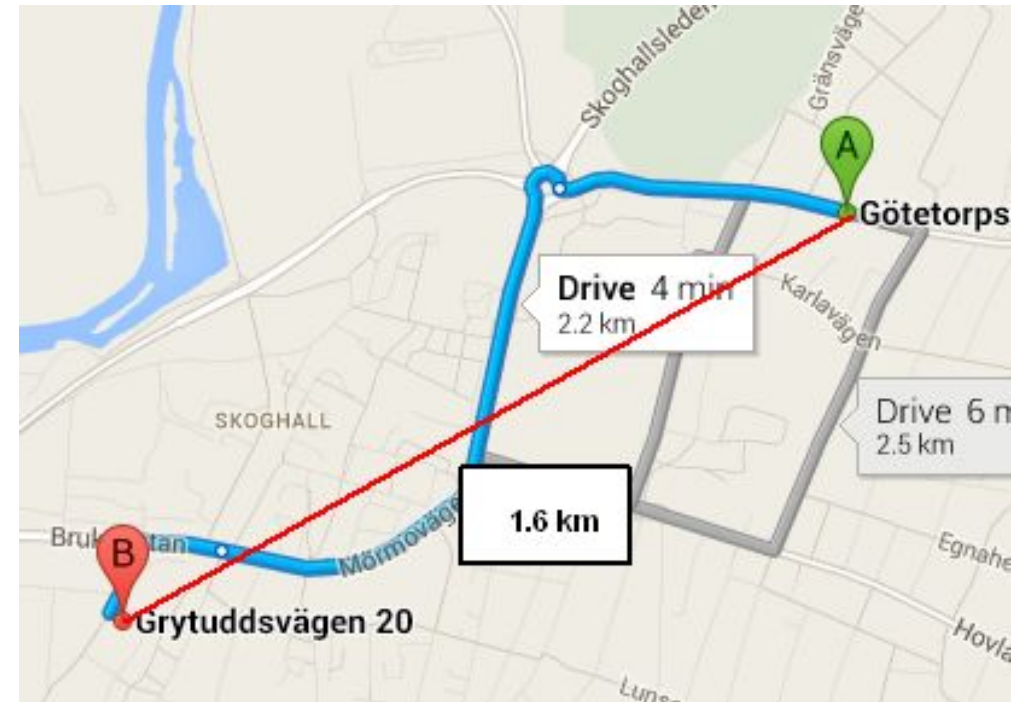
- Exponential complexity: $O(b^d)$.

□ **Space complexity**

- Exponential complexity (similar to other graph search algorithms).

Condition for Optimality: Admissibility

- A* search is **cost-optimal** if the heuristic $h(n)$ is admissible.
- An **admissible heuristic** is one that **never overestimates** the cost to reach a **goal**.
- Example,
 - The straight-line distance h_{SLD} is an admissible heuristic.
 - The reason is that the shortest path between any two points is a straight line, so h_{SLD} cannot be an overestimate.

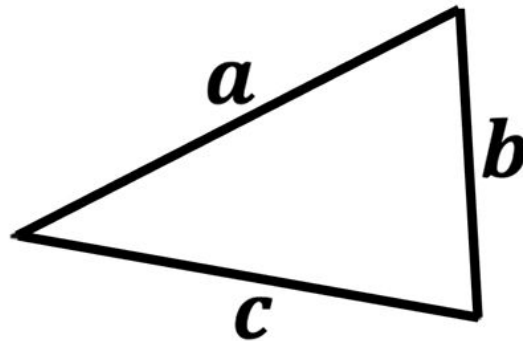


Condition for Optimality: Consistency

- A* search is **cost-optimal** if the heuristic $h(n)$ is **consistent**.
- **Consistency**, sometimes called **monotonicity**, is a slightly stronger condition than admissibility.
- An heuristic $h(n)$ is consistent if
$$h(n) \leq c(n, a, n') + h(n')$$
- In other words, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the action cost c of getting to n' plus the estimated cost of reaching the goal from n' .
- This is a form of the general **triangle inequality**.

Condition for Optimality: Consistency

- The **triangle inequality** states that for any triangle, the sum of the lengths of any two sides must be greater than or equal to the length of the remaining side.

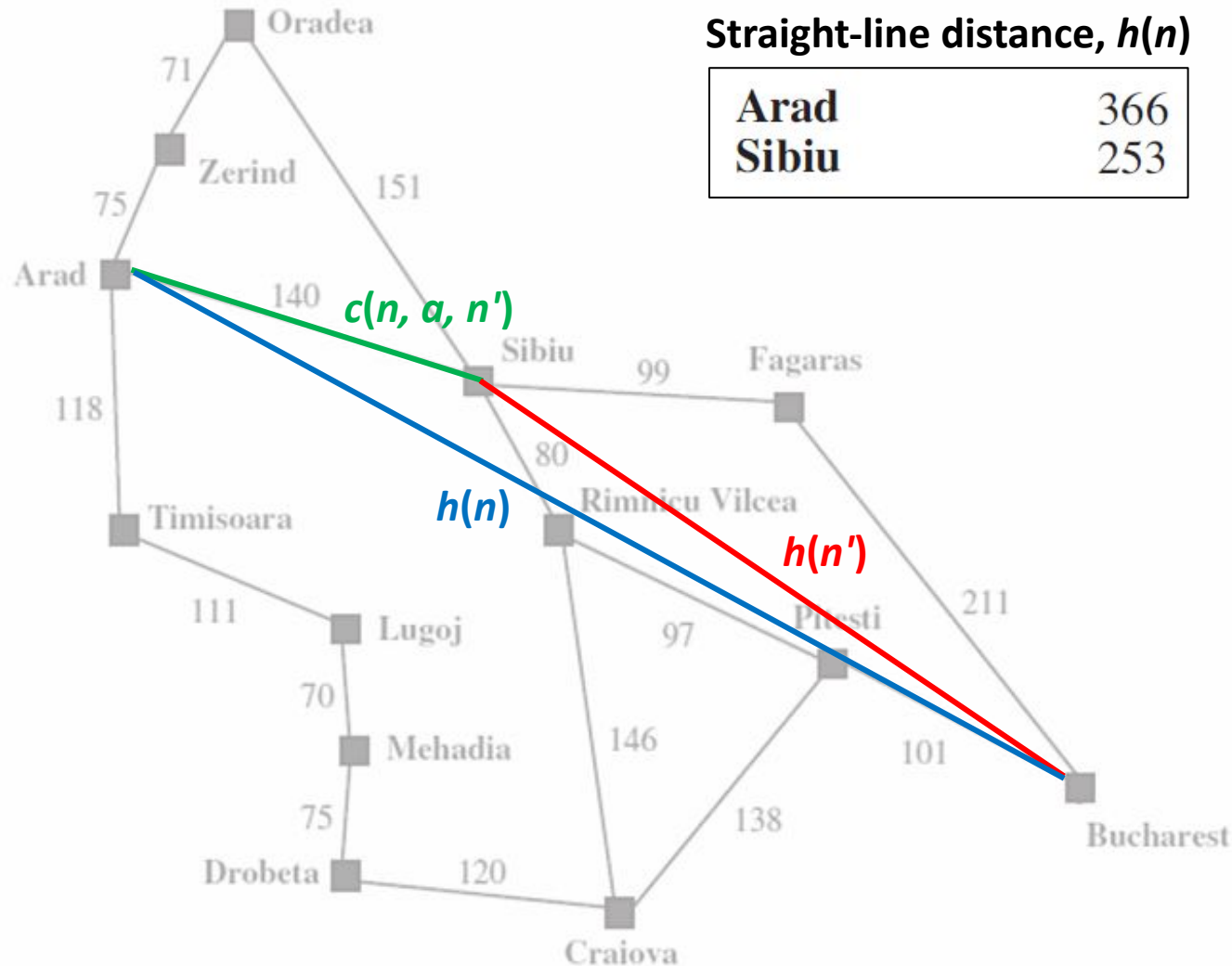


$$a + b > c$$

$$a + c > b$$

$$b + c > a$$

Condition for Optimality: Consistency



$$h(n) \leq c(n, a, n') + h(n')$$

- Assuming that $n = \text{Arad}$ and $n' = \text{Sibiu}$, then

$$h(n) = 366$$

$$c(n, a, n') = 140$$

$$h(n') = 253$$

- $\therefore h_{SLD}$ is a consistent heuristics

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 3: Solving Problems by Searching (page 98 to page 107)

The End